

Real-world (a cold interview on your own project)

— Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `project_walkthrough.md` : explaining your own project to someone who has never seen it is asked in nearly every AI engineering interview. The four beats — problem, shape, one decision, one limitation — give that explanation a structure you can reuse for any project. **If not:** you'd list features, and the interviewer would never hear *why* you built it that way.

Write your own answer first, in `practice/project_walkthrough.md` , then compare.

Model answer for: "Interview me about Project 4."

Basic Version

Approach 1 — the plain answer

Story: all four beats, said plainly, in under two minutes — including a limitation, offered before anyone asks. **If not:** you'd spend the time on features and never reach the part interviewers remember.

"Project 4 is called ContentForge. It's for teams that need short researched briefs written — someone gives it a task, and it researches it, works out what the findings mean, writes a draft, and has the draft reviewed before it's done.

It has five agents. One is a supervisor that decides what happens next — it looks at what's already in the shared state and sends the work to the right specialist: a researcher, an analyst, a writer, or a reviewer.

The decision I'd point to: I used a supervisor instead of a fixed research-then-write order, because the reviewer can reject a draft and send it back to the writer with its feedback — and a plain straight line can't loop back like that. To stop it looping forever, there's a limit: after 3 rejected drafts it stops and reports that the agents couldn't agree.

The thing I'd improve: the supervisor's rules always send every task through research first, even when the task already contains the facts. That wastes a paid research step. I'd let the supervisor skip stages that aren't needed."

This covers all four beats plainly, in under two minutes, and volunteers a real limitation without being asked — what a nervous first-timer says when they genuinely know their own project but haven't polished the vocabulary yet.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a structured answer

Story: the same four beats with the real mechanism and trade-off behind each one — the decision *and* what it cost. **If not:** the design would sound picked, not weighed.

"ContentForge solves content production for teams that need researched, reviewed briefs without one person running every step by hand.

It's Doc11's supervisor pattern with five agents: a supervisor that owns routing, a research agent that gathers findings, an analysis agent that works out what they mean, a writer that drafts from the analysis, and a reviewer that accepts or rejects the draft. Each agent writes its output into one shared state, and the supervisor reads that state after every step to pick the next agent — using `Command` for each hand-off.

The decision worth calling out: a supervisor instead of a fixed sequence, because review isn't a one-way gate — a rejected draft has to go *back* to the writer with the reviewer's feedback, and a fixed pipeline has no clean way to do that. The loop is bounded: after 3 rejections the run ends with a clear "couldn't agree" report and the full feedback history, instead of cycling forever. The trade-off I accepted: you can't read the code top to bottom and know the order anymore — you trace the supervisor's rules and the routing log, which records *why* each hop happened.

The honest limitation: the supervisor is rule-based, so every task goes through research first, even one that already includes the facts — a wasted, paid step. Given more time, I'd switch that one decision to Doc11's LLM-judged routing, and measure whether the saved research calls are worth the extra routing call."

Why this answer works: it's this document's Normal-depth rung, done properly — it opens with the user-facing problem, not the tech stack, which is what shapes an interviewer's first impression. It names the pattern *and* the specific requirement that pattern satisfies (backward routing when the reviewer rejects a draft), which is what "requirements → architecture is matching, not inspiration" (Doc16) actually looks like in speech. It states a real trade-off it accepted, not just a benefit, showing the choice was weighed, not just picked. And it volunteers the limitation with a concrete fix, unprompted.

What the Basic Version misses: the Basic Version's four beats are all correct, but each one stays at the surface level ("a supervisor that decides what happens next," "after 3 rejected drafts it stops") — Approach 1 names the *specific mechanism* underneath each beat (routing over shared state with

Command , the revision cap and its report, a hard trade-off in static reasoning) instead of describing the shape of the idea alone.

Approach 2 — when the interviewer pushes back

Story: strong interviewers challenge your decision and then switch projects to see if the structure is real. This practices conceding what's true, defending what's still true, and using the same four beats on a project you didn't rehearse. **If not:** the structure would work only for the one project you prepared.

Curveball 1 — the challenge, right after the Intermediate answer's "one decision": *"Isn't a full supervisor added complexity for one edge case? A fixed pipeline with a single retry loop around the review step would handle 'send it back to the writer' too, and it's simpler to reason about."*

Model response: "That's fair, and for exactly one back-edge, a fixed pipeline with a single conditional retry loop probably would be enough — I'll concede that directly. The reason I went with a supervisor anyway is that review wasn't the only place I expected a back-edge to show up. If a future specialist — say, an SEO-compliance check, or a tone-consistency pass — also needs the ability to send work backward to a different stage, a fixed pipeline needs a new hard-coded loop for every one of those, and they start interacting with each other in ways that get hard to follow. A supervisor already generalizes to that without new wiring each time. So the honest framing is: it's a bet on the pipeline growing more conditional paths over time, not a requirement the very first version strictly needed. If I were being graded purely on the requirements as they stood on day one, a fixed retry loop was probably the simpler, equally correct choice — the supervisor is me trading some debuggability now for not having to re-architect later if that bet pays off."

Curveball 2 — the cold project switch: *"Interesting — tell me about a different project instead, one where something went wrong."*

Model response, applying the same four-beat shape on the spot to a different project (Doc08's RAG pipeline, as an example): "Sure — the RAG project from Doc08. The problem was letting support staff ask questions against a large internal doc set instead of searching manually. The shape: documents get chunked and embedded into a vector store, a retriever pulls the top matches for a query, and a generation step answers using only those matches. The decision worth calling out: I set the retriever to a fixed top-k of 5 chunks, instead of a similarity-score cutoff, because it made latency and cost predictable per query — the trade-off was that on questions where the real answer needed 8 relevant chunks, it silently returned an incomplete answer instead of failing loudly. That's actually the thing that went wrong: it took a user complaint, not a metric, to catch it, because a confidently-worded partial answer doesn't look broken. The fix I added afterward was a similarity-score floor combined with the top-k cap, so it stops early on a strong single match but keeps pulling past 5 when the top results are all weakly similar to each other."

Why this answer works: Curveball 1's response concedes the true part of the challenge immediately instead of defending everything, then reframes the decision as a bet tied to a specific future need rather than a requirement that already existed — which is a more honest and more

senior answer than insisting the supervisor was obviously correct from day one. Curveball 2 shows the four-beat structure is a genuine reusable habit, not a script memorized for one project — and it volunteers a real failure ("it took a user complaint to catch it") without being asked twice, which is exactly the instinct Doc14's debugging material is checking for.

What a weaker answer misses: on Curveball 1, a weaker answer either fully concedes ("yeah, you're right, I over-engineered it") — abandoning a decision that had a real, specific justification the interviewer just hadn't heard yet — or argues without conceding anything, which reads as unable to take feedback. On Curveball 2, a weaker answer either can't produce the four beats for an unprepared project at all, or gives a plain list of features with no problem statement, decision, or limitation — revealing the structure was memorized per-project rather than genuinely internalized.

Which one should you actually give in a real interview? Open with the Basic Version's plain problem statement — that earns the interviewer's attention and trust in the first 15 seconds, regardless of their technical depth. Let your explanation rise naturally into Approach 1's vocabulary and trade-off language as you go. Don't pre-load Approach 2's pushback response or a second project before anyone asks — but when a challenge does land, concede what's true first, then defend the specific thing that's still true; and keep the four-beat shape general enough in your head that it works for any project you're asked about, not just the one you rehearsed tonight.